Auditoría general de la web YURA

Sistema: yura-platform (Laravel 12.58, PHP 8.2) · **Fecha:** 18-10-2026 · **Alcance:** código de la rama `feature/api-app-v1`, configuración local, dependencias y servidor Apache local.

Método: lectura del código, ejecución de la suite de pruebas, pruebas de sonda desechables, `composer audit` y consultas HTTP al servidor (solo códigos y encabezados; no se leyó el contenido de ningún archivo sensible).

---

1. Resumen ejecutivo

La aplicación está bien construida en lo funcional: autorización por rol y por propiedad en las notas, validaciones estrictas, sin salida HTML sin escapar y sin SQL concatenado. **El riesgo grave no está en el código sino en cómo se sirve:** el servidor Apache entrega por HTTP el archivo `.env`, la base de datos SQLite, el log de errores y muestra el listado de carpetas. Junto con `APP_DEBUG=true`, eso expone claves, datos y rutas internas a cualquiera que llegue al servidor.

Área	Estado	Comentario
Exposición del servidor	**Crítico**	Archivos sensibles descargables y listado de carpetas activo
Dependencias	**Alto**	54 avisos de seguridad en 15 paquetes (1 crítico, 17 altos)
Cuentas y contraseñas	**Alto**	Contraseñas conocidas en los seeders
Autorización en rutas	Medio	Una ruta antigua deja que el robot modifique cualquier estudiante
Integridad de datos	Medio	Historial de notas duplicado e inexacto; base de datos desalineada
Auditoría	Medio	Solo registra la administración de usuarios
Calidad de código	Bajo	Controladores grandes, columnas duplicadas por compatibilidad

Lo primero que hay que hacer (en orden): 1) dejar de servir la carpeta del proyecto completa, 2) poner `APP_DEBUG=false`, 3) actualizar dependencias, 4) cambiar las contraseñas conocidas.

---

2. Cómo trabaja la web

Tecnología: Laravel 12.58 sobre PHP 8.2, Blade con AdminLTE, API con Sanctum, Pest/PHPUnit para pruebas. Sesiones, caché y colas en la base de datos.

Tamaño: 19 controladores web y 9 de API, 15 modelos, 46 migraciones, 48 vistas Blade, unas 6.500 líneas de PHP en `app/`, 120 rutas, 102 pruebas automáticas.

Roles (8): administrador, académico, profesor, padre, madre, tutor, estudiante y robot.

Módulos y quién los usa

Módulo	Qué hace	Roles
Usuarios y vínculos familiares	Altas, edición, suspensión, borrado recuperable, relación familia–estudiante	Administrador (académico en vínculos)
Estudiantes	Registro, ficha, fotos, exportación a PDF y Excel	Administrador, académico, profesor; familia ve los suyos
Estructura académica	Cursos, paralelos, materias, asignaciones docentes	Administrador y académico
Cuaderno pedagógico	Notas por trimestre, materia y tipo (actividad, examen); historial de cambios	Profesor (con asignación); familia en lectura
Asistencia	Presente, falta, atraso, licencia por materia y fecha	Profesor
Actividades del robot	El profesor crea actividades grupales; el cerebro las reclama y devuelve notas (contrato v1.2)	Profesor, robot
Portal público	Documentos, servicios y búsqueda	Sin ingreso
API para la app	Familia, profesor (lectura) y administrador	Por rol, con token
Auditoría	Registro de acciones	Administrador

Flujo principal: el profesor califica en el cuaderno → la nota se guarda con su historial → la familia ve promedios y asistencia en su panel (los mismos números que la app, por un servicio compartido) → el profesor puede lanzar una actividad al robot, que devuelve la nota del grupo.

Protección de rutas: de 120 rutas, 109 exigen sesión o token y 11 son públicas por diseño (portal, login, logout, selector de idioma, ingreso por API). La autorización fina se hace dentro de los controladores (no hay Policies).

---

3. Hallazgos

Críticos

ID	Hallazgo	Evidencia	Corrección
A1	**El servidor entrega archivos sensibles.** `/yura-platform/.env` (1.223 bytes), `database/database.sqlite` (274 KB), `storage/logs/laravel.log`, `composer.json`, `vendor/…`, el diccionario `.db` y las carpetas `tinker_scripts/` y `consigna/` responden 200. El listado de carpetas está activo y no hay `.htaccess` en la raíz.	Consultas HTTP (código y tamaño). No se leyó el contenido.	Que el servidor sirva solo `public/` (VirtualHost con `DocumentRoot` en `public`), `Options -Indexes`, bloquear archivos que empiezan por punto, mover los `.db` fuera de htdocs. **Sin corregir:** es configuración de Apache.
A2	**`APP_DEBUG=true` y `APP_ENV=local`.** Cualquier error muestra trazas con rutas y datos.	Lectura de las dos claves del `.env`.	`APP_DEBUG=false`, `APP_ENV=production`, y regenerar `APP_KEY` si el `.env` pudo descargarse. **Sin corregir.**

Altos

ID	Hallazgo	Evidencia	Corrección
B1	**54 avisos de seguridad en dependencias** (1 crítico, 17 altos, 25 medios). Crítico: `phpoffice/phpspreadsheet` (CVE-2026-45034). Altos relevantes: `laravel/framework` (inyección CRLF en la regla de correo), `dompdf` (lectura de archivos por SVG, usado en los PDF), `maatwebsite/excel` (escribe fuera del directorio configurado), `guzzle`, `symfony/http-kernel`, `symfony/mime`.	`composer audit`	`composer update` dentro de las versiones permitidas y volver a pasar la suite. **Sin corregir.**
B2	**Contraseñas conocidas en los seeders:** `password123` para administrador, profesores, gestor y familias; `robot-password-123` para el robot. El seeder del robot imprime su token en la consola.	Lectura de `SchoolPilotSeeder` y `RobotUserSeeder`; el ingreso del administrador con `password123` funciona en la base de prueba.	No ejecutar los seeders en producción; cambiar de inmediato las contraseñas de esas cuentas si ya existen en MySQL; rotar el token del robot. **Sin corregir.** No verifiqué si tu MySQL real las tiene.
B3	**Ruta antigua `POST /api/students/{id}/update-score`:** la comprobación excluye solo familias y estudiantes. Un token de robot y uno de académico modifican la nota de cualquier estudiante.	Prueba de sonda: robot 200, académico 200, administrador 200, profesor 403 (en un estudiante ajeno).	Lista blanca de roles (administrador y profesor dueño). **Sin corregir.**

Medios

ID	Hallazgo	Evidencia	Corrección
C1	**La auditoría solo cubre usuarios.** Notas, asistencia, estudiantes y actividades del robot no dejan registro. Antes de esta rama no se escribía nada.	Búsqueda de usos de `AuditLog`	Registrar también cambios de notas, asistencia y estudiantes.
C2	**Historial de notas inexacto.** Al editar una nota desde el formulario, el controlador lee `getOriginal('score')` después de guardar y crea una segunda fila de historial; el observador ya había creado la correcta.	Lectura del código (`GradeController@update`). El observador se verificó con una prueba: una edición de 50 a 90 deja una fila correcta.	Dejar el registro solo al observador y pasarle el motivo.
C3	**Base de datos desalineada.** El `.env` apunta a SQLite, pero tú trabajas con MySQL; la auditoría anterior encontró la MySQL sin las tablas del robot. El código convive con columnas duplicadas (`role/rol`, `status/estado`) y consulta el esquema en tiempo de ejecución.	`.env`, modelos	Decidir una sola base y migrar; retirar las columnas heredadas. No verifiqué el estado actual de tu MySQL.
C4	**Sin HTTPS ni configuración de CORS.** Los tokens viajan en claro dentro de la red.	Ausencia de `config/cors.php` y de redirección	HTTPS en producción antes de abrir la app.

ID	Hallazgo	Evidencia	Corrección
C5	**Sin Policies:** la autorización depende de comprobaciones repartidas por los controladores. Hay 10 rutas de escritura sin filtro de rol en la ruta. Revisé una por una las sensibles (`force` , `restore` , creación de estudiantes por API): están protegidas dentro del controlador, salvo B3.	Análisis de rutas y lectura de controladores	Centralizar en Policies; hace menos probable una regresión.

Bajos

ID	Hallazgo
D1	Controladores grandes: `StudentController` 639 líneas, `GradeController` 468, mezclan validación, permisos y reglas.
D2	Archivos de trabajo en la raíz del proyecto (diccionario `.db` , `consigna/` , `tinker_scripts/` con salidas); además quedan expuestos por A1.
D3	Sesiones, caché y colas en la base de datos: rinde peor que archivos o Redis. No medido.
D4	No se midieron consultas repetidas (N+1) ni tiempos de respuesta.

---

4. Lo que está bien (verificado)

- **Sin XSS por salida sin escapar:** ninguna vista usa `{!! !!}` para datos del usuario.
- **Sin inyección SQL:** las 6 expresiones SQL crudas usan valores fijos o parámetros.
- **Asignación masiva controlada:** los modelos declaran `$fillable` .
- **Subida de fotos:** solo jpg, png y webp, máximo 2 MB, con nombre generado por Laravel.
- **Límite de intentos** en el ingreso web y en el de la API (5 por minuto).
- **Autorización de notas por propiedad:** el profesor solo califica con una asignación docente; la familia solo ve a sus hijos con vínculo activo.
- **Borrado recuperable** (soft delete) en usuarios y estudiantes; el borrado definitivo es solo del administrador.
- **Contraseñas fuertes** exigidas en los formularios de usuarios.
- **Pruebas:** 102 pruebas automáticas en verde.

---

5. Ya corregido en la rama `feature/api-app-v1` (pendiente de unir a `develop`)

- Vínculo familiar inactivo o sin permiso de ver ya no da acceso, en web y API.
- Tokens de la app con vencimiento (familia 30 días, administrador 7); el del robot no vence.
- Rutas de la app con control de rol; el profesor solo lee.
- Endpoint antiguo `POST /api/v1/robot/activity` solo para el rol robot.
- Límite de intentos del ingreso por cuenta e IP; usuarios suspendidos pierden el acceso con su token.
- Administración de usuarios con reglas compartidas: nadie se suspende, borra ni cambia el rol a sí mismo; siempre queda un administrador activo; las sesiones se cierran al suspender, borrar o

cambiar rol o contraseña; todo en transacción y con auditoría.
- Se identificó que las 52 pruebas originales fallaban en esta máquina por la URL del `.env`; ya se fijó en `phpunit.xml`.

6. Lo que no se pudo verificar

- El estado real de tu **MySQL** (el `.env` apunta a SQLite).
- Si las cuentas de los seeders existen en tu base real.
- Rendimiento y consultas repetidas.
- Comportamiento en producción con HTTPS, y copias de seguridad.
- La app en un celular real con todos los roles.

7. Plan de corrección sugerido

Prioridad	Acción	Esfuerzo
1	A1 y A2: servir solo `public/`, quitar el listado de carpetas, `APP_DEBUG=false`; si el `.env` pudo descargarse, regenerar `APP_KEY` y cambiar claves de base de datos	1 h
2	B2: cambiar las contraseñas de las cuentas sembradas y rotar el token del robot	30 min
3	B1: `composer update` y volver a pasar la suite	1–2 h
4	B3: restringir `update-score`, con prueba	30 min
5	C2 y C1: historial de notas correcto y auditoría ampliada	2–3 h
6	C3: una sola base de datos y migración limpia	2–4 h
7	C4, C5, D1: HTTPS, Policies y refactor	varias sesiones